

AFL Assistant — Executive Report

Domain-locked chat, retrieval & prediction assistant — capstone summary

Product Goal

Ship a single AFL assistant that can answer factual questions, retrieve real statistics, and make match/player predictions — without ever confusing the three. The core product requirement is trust: a prediction must always be framed as a probability with a disclaimer, never presented as a guaranteed outcome, and the assistant must stay strictly within AFL scope even under adversarial prompting.

Architecture

Built as a LangGraph state machine rather than one free-form agent, specifically because the general chat/retrieval agent (Gemini-backed, with five retrieval tools over historical match/player data) has **no prediction capability at all**. A single agent handed a prediction-shaped query would have to either refuse incorrectly or hallucinate a winner from language-model “knowledge” — unacceptable for a stats-grounded product. The router intercepts prediction-shaped queries before they can reach that agent:

- **Router** — classifies each query as factual / retrieval / prediction (match or player) / off-topic / unsupported, using a deterministic rule-based classifier by default (a swappable Gemini-based classifier is available for better generalisation).
- **Prediction path** — resolves team names against the real dataset, calls fitted scikit-learn pipelines (Logistic Regression for match winner, Gradient Boosting for top disposal-getter), and validates the result before it can reach the user.
- **Chat/retrieval path** — delegates factual, retrieval, and off-topic queries entirely to the existing Day 3 agent rather than reimplementing that logic.
- **Validation & clarification** — an unresolved team name asks the user to clarify instead of guessing; a genuinely out-of-scope prediction (e.g. an exact score, or “who wins the Grand Final”) gets an honest “can’t do that” response, not a hallucination.
- **API/UI** — a FastAPI wrapper (structured JSON logging, real async timeouts, basic abuse-signal tracking) and a Streamlit chat UI sit on top for demoing and integration.

Evaluation Results

A 34-case suite spanning four categories currently passes 100% offline (router + real prediction models + a scripted stand-in for the live chat agent). This validates the graph’s structure — routing correctness, fail-closed behavior on unresolved entities, and multi-turn state handling — which is fully testable without external dependencies:

Category	Cases	Pass	What it covers
Factual Q&A routing	7	100%	Correct routing of AFL rules/history questions
Prediction sanity	9	100%	Probabilities move sensibly with matchup strength; reversed home/away; degenerate cases
Scope guardrails	11	100%	Off-topic refusal, unsupported-scope handling, 5 prompt-injection attempts
Multi-turn coherence	7	100%	Context across turns; recovery after a clarification round

Important caveat: the offline suite uses a scripted stand-in for the live Gemini chat agent (no API key in this evaluation environment). It validates plumbing, not live answer quality or live prompt-injection resistance — both must be re-verified against the real agent before launch.

Benchmark context: the match-winner model reaches 63.4% test accuracy vs. a 56.3% naive baseline (always predict the home team) — a real but modest ~7-point lift; AFL outcomes are genuinely hard to predict from

pre-match features alone. The top-player model's 63.0% top-5 hit rate is currently **below** an even simpler baseline (71.9%, "last week's leader repeats") — it is retained for its explainable, grounded reasoning rather than raw accuracy, and this should be stated plainly to stakeholders rather than glossed over.

Known Limitations

- **Data recency / no real fixture calendar.** Predictions use each team's latest known rolling state rather than a specific upcoming fixture with a real date/venue — "this week" is not tied to an actual schedule in the current data.
- **Model accuracy ceiling.** 63.4% match-winner accuracy is a real, useful signal, not a confident oracle; the top-player model does not currently beat its naive baseline.
- **Rule-based router has no coreference resolution** — a follow-up like "what about their stats" needs a named team, not a pronoun, unless the Gemini-based router variant is enabled.
- **Guardrail edge cases** are evaluated here against a stand-in agent; live verification against the real Gemini-backed agent is a required pre-launch step.

Recommended Next Steps

- Re-run the full evaluation suite against the live chat agent (real GEMINI_API_KEY) and treat that run's results as authoritative before launch.
- Source a real fixture calendar so predictions can be tied to actual upcoming matches, venues, and dates rather than "latest known state."
- Stand up the weekly data-refresh / monthly retraining loop described in the monitoring plan, with accuracy-drift alerting against the documented baselines.
- Either improve the top-player model past its naive baseline or reframe its role in the product as explanatory context rather than a headline prediction.
- Move the in-memory abuse-signal tracker to a shared store (e.g. Redis) before any multi-instance production deployment.

Bottom line

The system is structurally sound and honest about its own limits — predictions are always hedged, off-topic requests (including adversarial ones) are refused rather than answered, and the two areas where the model genuinely underperforms (top-scorer prediction vs. a naive baseline; no real fixture calendar) are surfaced here rather than hidden. It is ready for a live-agent verification pass and a staged rollout, not yet for an unmonitored production launch.

Full codebase, evaluation logs, monitoring checklist, and demo script accompany this report.